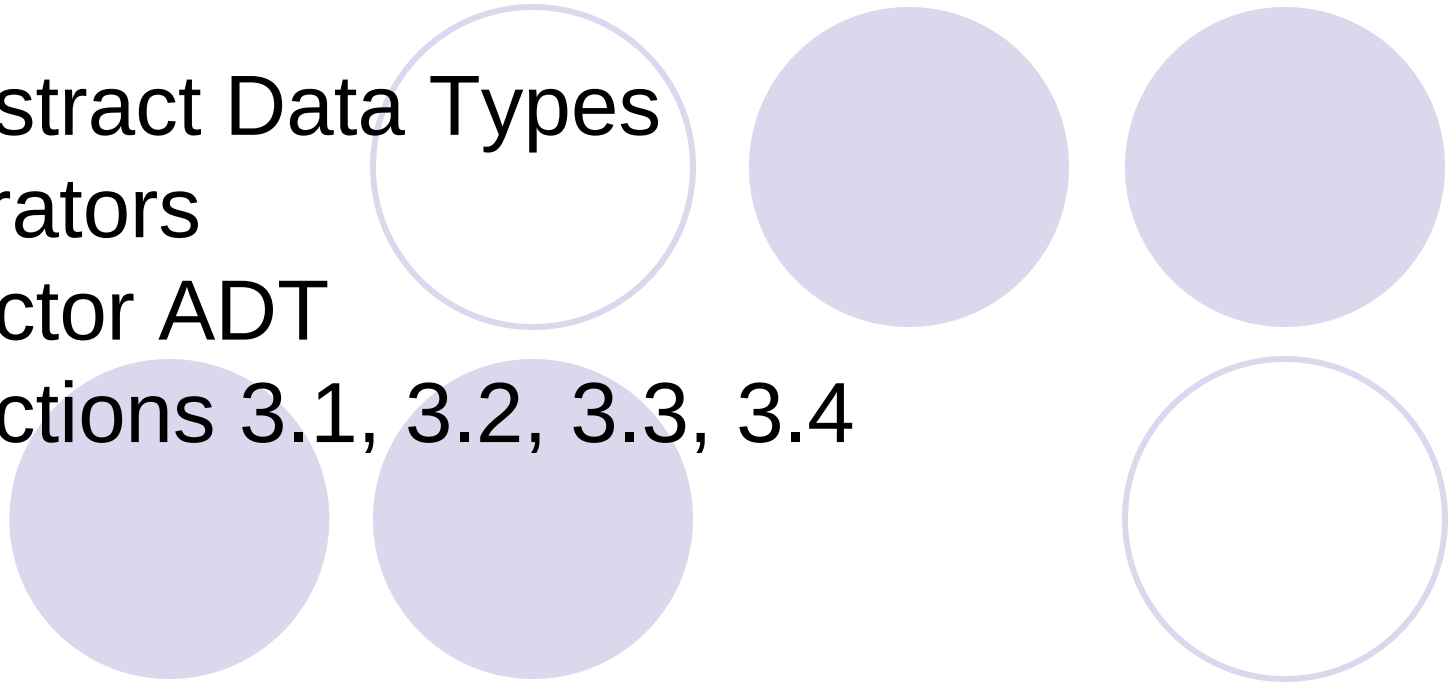
Abstract Data Types
Iterators
Vector ADT
Sections 3.1, 3.2, 3.3, 3.4



Abstract Data Type (ADT)

- High-level definition of data types
- An ADT specifies
  - A *collection* of data
  - A set of *operations* on the data or subsets of the data
- ADT does not specify how the operations should be implemented
- Examples
  - vector, list, stack, queue, deque, priority queue, table (map), associative array, set, graph, digraph

Standard Template Library (STL)

- The STL consists of:
  - Container: e.g. vector, list, queue, stack,…
  - Iterator
  - Algorithm
  - function object

Iterators

- A position is represented by Iterators in STL (similar to the pointer).
- Helps navigate through the items in a list.
- An example: iterator over `vector v`.

```
for (int i = 0; i != v.size(); i++ )
  cout << v[i] << endl;
```

Iterators (contd.)

- A generalized type that help in navigating any container
  - A way to initialize at the front and back of a list
  - A way to move to the next or previous position
  - A way to detect the end of an iteration
  - A way to retrieve the current value

- Examples:
  - Iterator type for **`vector<int>`** defined as
    - **`vector<int>::iterator itr;`**
  - Iterator type for **`list<string>`** defined as
    - **`list<string>::iterator itr;`**

Getting an Iterator

- Two methods in all STL containers
  - `begin()`
    - Returns an iterator to the first item in the container
  - `end()`
    - Returns an iterator representing the container
      (i.e. the position after the last item)

- Example:

```
for (int i = 0; i != v.size(); i++ )
   cout << v[i] << endl;
```

can be written using iterators as

```
for(vector<int>::iterator itr=v.begin(); itr!=v.end(); itr.???)
       cout << itr.??? << endl;
```

- What about `???`

Iterator Methods

- Iterators have methods
- Many methods use operator overloading
  - **`itr++ and ++itr`** ➔advance the iterator to next location
  - **`*itr`** ➔ return reference to object stored at iterator **`itr's`** location
  - **`itr1 == itr2`** ➔true if **`itr1`** and **`itr2`** refer to the same location, else false
  - **`itr1 != itr2`** ➔true if **`itr1`** and **`itr2`** refer to different locations, else false
- Previous example becomes

```
for(vector<int>::iterator itr= v.begin(); itr!= v.end(); itr++)
     cout << *itr << endl;
```

- Alternatively

```
vector<int>::iterator itr = v.begin( );
while( itr != v.end( ) )
   cout << *itr++ << endl;
```

Container Operations That Require Iterators

- iterator insert(iterator pos, const Object & x)
- iterator erase(iterator pos)
- iterator erase(iterator start, iterator end)

Iterator example: erase

```
1   template <typename Container>
2   void removeEveryOtherItem( Container & lst )
3   {
4       auto itr = lst.begin( );      // itr is a Container::iterator
5
6       while( itr != lst.end( ) )
7       {
8           itr = lst.erase( itr );
9           if( itr != lst.end( ) )
10              ++itr;
11      }
12  }
```

const_iterator

- Returns a constant reference for **`operator*`**
- So that a function does not try to modify the elements of a constant container object.
- Note that **`c.begin()`** and **`c.end()`** functions in the example return **`const_iterator`** type.

```
1  template <typename Container>
2  void printCollection( const Container & c, ostream & out = cout )
3  {
4      if( c.empty( ) )
5          out << "(empty)";
6      else
7      {
8          typename Container::const_iterator itr = c.begin( );
9          out << "[ " << *itr++;   // Print first item
10
11         while( itr != c.end( ) )
12             out << ", " << *itr++;
13         out << " ]" << endl;
14     }
15 }
```

The Vector ADT

Generic arrays

The Vector ADT

- Extends the notion of array by storing a sequence of arbitrary objects
- Elements of vector ADT can be accessed by specifying their index.

Vectors in STL

- Collection ➔ Elements of some proper type `T`
- Operations
  - **`int size()`** ➔ returns the number of elements in the vector
  - **`void clear()`** ➔ removes all elements from the vector
  - **`bool empty()`** ➔ returns true of the vector has no elements
  - **`void push_back ( const Object &x )`**
    - adds `x` to the end of the vector
  - **`void pop_back ( )`**
    - Removes the object at the end of the vector
  - **`Object & back ( )`**
    - Returns the object at the end of the vector
  - **`Object & front ( )`**
    - Returns the object at the front of the vector

Vectors in STL (contd.)

- More Operations
  - **`Object & operator[] ( int index )`**
    - Returns the object at location index (without bounds checking)
    - Both accessor and mutator versions
  - **`Object & at( int index )`**
    - Returns the object at location index (with bounds checking)
  - **`int capacity()`**
    - Returns the internal capacity of the vector
  - **`void reserve(int newCapacity)`**
    - Sets the new capacity of the vector
  - void **`resize(int newSize )`**
    - Change the size of the vector

Implementing Vector Class Template

- Vector maintains
  - A primitive C++ array
  - The array capacity
  - The current number of items stored in the Vector
- Operations:
  - Implement the Big-Five: Copy constructor, `operator=`, Destructor to reclaim primitive array.
  - All the other operators we saw earlier.

Vector Implementation

Figures 3.7 and 3.8